

Module 7

Vision Transformers and Advanced Visual Representation Learning

Selina Ochukut

Module Overview

This module extends the attention and Transformer machinery from Module 3 into computer vision. You will explore the **Vision Transformer (ViT)**, which treats an image as a sequence of patches and processes it with an otherwise-unmodified Transformer encoder, and examine the broader landscape of **visual representation learning** it belongs to: methods for learning general-purpose image features from data, with or without labels. We cover **self-supervised and contrastive learning** methods that learn from unlabeled images, **multimodal foundation models** such as **CLIP** that learn joint image-text representations from web-scale data, and **Attention Rollout**, a method for interpreting what a trained Vision Transformer is actually looking at.

Expected Learning Outcomes

By the end of this module, students should be able to:

1. **Explain** the principles of Vision Transformers (ViTs), self-supervised learning, contrastive learning, and CLIP.
2. **Apply** the Vision Transformer patch-embedding pipeline.
3. **Analyze** the training objectives of major self-supervised and contrastive learning methods.
4. **Evaluate** the effectiveness of Vision Transformers, self-supervised learning approaches, and CLIP for computer vision tasks.

1 Introduction to Vision Transformers and Visual Representation Learning

1.1 From Convolutions to Attention

For nearly a decade, convolutional neural networks (CNNs) were the dominant architecture for computer vision, owing to two built-in **inductive biases** well-suited to images:

- **Locality:** A convolutional filter only looks at a small local neighborhood of pixels at a time, reflecting the fact that nearby pixels are usually more related than distant ones.
- **Translation equivariance:** The same filter weights are applied at every spatial location, so a shifted input produces a correspondingly shifted output – a natural fit for recognizing an object regardless of where it appears in the image.

Key Idea

These inductive biases are a double-edged sword: they make CNNs highly **data-efficient** (a useful prior is built directly into the architecture, so less data is needed to discover it from scratch), but they also *limit* what the network can easily represent – a single convolutional layer can only relate nearby pixels, and modeling long-range dependencies (e.g., relating one corner of an image to another) requires stacking many layers to gradually grow the receptive field. The **Vision Transformer (ViT)** (Dosovitskiy et al., 2020) asks: what if we remove these hand-built biases almost entirely, and let a Transformer – with self-attention’s inherent ability to relate *any* two positions in a single layer (Module 3) – learn whatever spatial structure is useful directly from data?

1.2 Video: Vision Transformers

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/Hnsrm1ezI3g>



1.3 The Central Trade-off: Inductive Bias vs. Scale

Key Idea

ViT’s original paper reported that, when trained on a mid-sized dataset like ImageNet-1k alone, a pure Transformer underperforms a comparable CNN – consistent with the idea that the CNN’s built-in locality/equivariance biases are genuinely useful when data is limited. However, when pretrained on much larger datasets (tens to hundreds of millions of images), ViT matches or exceeds CNN performance, since with enough data, the network can learn useful spatial priors on its own, and having *fewer* hard-coded assumptions leaves *more* capacity available to discover patterns a human-designed architecture might not have anticipated. This established a now-common theme in modern deep learning: architectures with weaker inductive biases but higher capacity tend to win out *given enough data and compute*, a pattern also seen when Transformers overtook RNNs in NLP (Module 3).

1.4 What Is Visual Representation Learning?

Definition

Visual representation learning is the study of how to learn general-purpose image embeddings – vector representations of images (or image patches/regions) – that capture useful visual and semantic structure, in a way that transfers well to *many* downstream tasks (classification, detection, segmentation, retrieval, captioning), rather than being tailored to a single task from the start.

Key Idea

A recurring theme of this module is that *how* a representation is learned – supervised on labeled classes, self-supervised on unlabeled images alone, or contrastively aligned with natural-language descriptions – has a major effect on what kind of downstream tasks the resulting representation transfers well to, and how much labeled data (if any) is needed to adapt it. This module surveys the major paradigms in roughly the order they build on one another: architecture (ViT), self-supervised/contrastive pretraining objectives, and multimodal foundation models.

2 Patch Embeddings and the Transformer Encoder Architecture

The core challenge in applying a Transformer to images is representational: Module 3’s Transformer operates on a *sequence* of token vectors, but an image is a 2D (or 3D, with channels) grid of pixels. ViT’s solution is to convert the image into a sequence of **patch embeddings** and then apply an otherwise-standard Transformer encoder.

2.1 Patchify and Linear Projection

Definition

Given an input image $x \in \mathbb{R}^{H \times W \times C}$, ViT first splits it into a grid of $N = \frac{HW}{P^2}$ non-overlapping square **patches**, each of size $P \times P \times C$ (e.g., $P = 16$). Each patch is flattened into a vector of length P^2C , and linearly projected into a D -dimensional **patch embedding**:

$$z_i = E \cdot \text{flatten}(x_i) + b, \quad i = 1, \dots, N$$

where $E \in \mathbb{R}^{D \times P^2C}$ is a learned projection matrix (shared across all patches).

Key Idea

This patchify-and-project operation is mathematically identical to a single convolutional layer with kernel size $P \times P$ and stride P (a non-overlapping “strided” convolution) – ViT’s very first operation is, in fact, a small convolution, and the “pure Transformer, no convolutions” framing really refers to everything *after* this initial patch embedding step. Choosing a smaller patch size P increases the number of patches N (finer spatial resolution, but quadratically more expensive self-attention, since attention costs $O(N^2D)$ as established in Module 3), while a larger P is cheaper but coarser – a direct, tunable trade-off between resolution and compute.

2.2 The Classification Token

Definition

Following the BERT convention from NLP, ViT prepends a special, learnable **classification token** embedding z_{cls} to the sequence of patch embeddings, so the input to the Transformer encoder is $[z_{cls}, z_1, \dots, z_N]$. After passing through the full encoder, the *final* representation of this token, $z_{cls}^{(L)}$, is used as the aggregate, whole-image representation for downstream classification.

Key Idea

Because self-attention lets every token attend to every other token, the [CLS] token can, over the course of the encoder’s layers, gather and aggregate information from every patch in the image into a single vector – serving the same role that global average pooling serves at the end of a CNN, but learned and content-dependent (via attention) rather than a fixed, uniform average.

2.3 Positional Embeddings

Self-attention alone is permutation-equivariant and has no notion of *where* each patch is located in the image. ViT adds a learned **positional embedding** E_{pos} to each patch embedding (and to the [CLS] token) before the first encoder layer:

$$z_0 = [z_{cls} + E_{pos}^{(0)}, z_1 + E_{pos}^{(1)}, \dots, z_N + E_{pos}^{(N)}]$$

Key Idea

Unlike the fixed sinusoidal positional encoding often used in NLP Transformers, ViT typically uses *learned* 1-D positional embeddings (one per patch index, with no explicit 2-D spatial structure hard-coded), and the model is left to discover any useful 2-D spatial relationships (e.g., “adjacent row/column”) on its own during training – another example of ViT’s general philosophy of minimizing hard-coded assumptions in favor of letting the data determine what structure matters.

2.4 The Transformer Encoder

Definition

The sequence of (position-augmented) patch embeddings is passed through L standard Transformer encoder layers, exactly as defined in Module 3: each layer applies multi-head self-attention followed by a position-wise MLP, each wrapped in a residual connection and layer normalization:

$$z'_\ell = z_{\ell-1} + \text{MSA}(\text{LN}(z_{\ell-1})), \quad z_\ell = z'_\ell + \text{MLP}(\text{LN}(z'_\ell))$$

Key Idea

ViT uses the **Pre-LN** convention (layer normalization applied *before* each sublayer, rather than after, as discussed in Module 3), which was found to produce more stable training, particularly important for very deep ViT variants. Aside from this convention and the patch-embedding front-end, ViT’s encoder is architecturally identical to a text Transformer encoder – underscoring that, once an image has been converted into a sequence of vectors, the rest of the machinery from Module 3 transfers directly, with no vision-specific modification required.

2.5 Classification Head and Model Variants

A final MLP head (typically a single linear layer during fine-tuning, or an MLP with one hidden layer during pretraining) maps the final [CLS] representation $z_{cls}^{(L)}$ to class logits. ViT is commonly instantiated at several standard sizes (ViT-Base, ViT-Large, ViT-Huge), differing in embedding dimension D , depth L , and number of attention heads – directly analogous to how BERT/GPT-style text Transformers are scaled.

2.6 Data-Efficient Variants: DeiT

Key Idea

Since ViT’s lack of convolutional inductive bias makes it data-hungry, **DeiT** (Data-efficient Image Transformers, Touvron et al., 2021) showed that a ViT can be trained effectively on a much smaller dataset (ImageNet-1k alone) using stronger data augmentation, regularization,

and a **distillation token**: an additional special token trained to match the predictions of a pretrained CNN “teacher” model via knowledge distillation. This demonstrates that ViT’s data-hungriness is a property of the *training recipe*, not an immovable property of the architecture itself.

3 Self-Supervised and Contrastive Learning Methods

Labeled data is expensive to collect at scale, while unlabeled images are abundant. **Self-supervised learning (SSL)** designs a **pretext task** – a task with automatically-generatable “labels” derived from the data itself – so a network can learn useful representations from unlabeled images alone, which can then transfer to downstream tasks via fine-tuning or linear probing.

3.1 The Contrastive Learning Framework

Definition

Contrastive learning trains an encoder so that representations of two different **augmented views** of the *same* image (a **positive pair**) are pulled close together in embedding space, while representations of views from *different* images (**negative pairs**) are pushed apart.

Key Idea

The pretext task here is implicit: an encoder cannot easily learn to recognize that two heavily and differently augmented crops (e.g., different crop regions, color distortions, blurring) came from the same underlying image unless it learns to focus on the image’s actual semantic content (the object present), rather than low-level, augmentation-sensitive pixel statistics – which is exactly the kind of representation that tends to transfer well to downstream recognition tasks.

3.2 SimCLR

SimCLR (Chen et al., 2020) is a conceptually simple, influential contrastive framework:

1. Given an image, generate two randomly augmented views (random crop, color jitter, Gaussian blur, etc.).
2. Pass both views through a shared encoder $f(\cdot)$, then a small **projection head** MLP $g(\cdot)$, producing embeddings z_i, z_j .
3. Within a training batch of N images (yielding $2N$ augmented views), treat the other view of the same image as the single positive, and all

other $2(N - 1)$ views in the batch as negatives, minimizing the **NT-Xent** (normalized temperature-scaled cross-entropy) loss:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity and τ is a temperature hyperparameter.

Key Idea

SimCLR’s loss is directly analogous to a softmax classification loss where the “correct class” is the matching positive view among $2N - 1$ candidates – and because *every other view in the batch* serves as a negative, SimCLR’s performance is strongly dependent on using a **very large batch size** (thousands of images) to provide enough negatives for the contrastive signal to be informative, which is computationally demanding.

3.3 MoCo: Decoupling Negatives from Batch Size

Definition

MoCo (Momentum Contrast, He et al., 2020) addresses SimCLR’s large-batch requirement by maintaining a large **queue** of negative embeddings computed from *previous* batches, decoupling the number of available negatives from the current batch size. To keep these queued embeddings consistent with the current encoder (since the encoder is continually changing), MoCo uses a **momentum encoder**: a second copy of the encoder whose weights θ_k are updated as a slow exponential moving average of the main (query) encoder’s weights θ_q : $\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$, with m close to 1 (e.g., 0.999).

Key Idea

The momentum encoder is a direct conceptual cousin of DQN’s target network: both address the problem of needing a *slowly-changing, stable* reference to compare against, while the main network being trained changes rapidly at every step. This lets MoCo maintain a queue of tens of thousands of negatives that remain reasonably consistent with each other, without needing an enormous batch size.

3.4 BYOL: Contrastive Learning Without Negatives

Key Idea

BYOL (Bootstrap Your Own Latent, Grill et al., 2020) makes a striking claim: negative pairs are not actually necessary at all. BYOL trains an **online network** to directly *predict* the output of a **target network** (a momentum-updated copy of the online network, as in MoCo) applied to a differently-augmented view of the same image – using only positive pairs. Naively, this setup risks **representation collapse** (the trivial solution of mapping every input to the same constant vector perfectly satisfies a positive-only matching objective), which BYOL avoids through a combination of architectural asymmetry (an extra **predictor** MLP on the online branch only) and the momentum target network, without any explicit negative examples or repulsive term in the loss.

3.5 Masked Image Modeling: MAE

Inspired directly by masked language modeling (Module 3’s BERT-style pretraining), **Masked Autoencoders (MAE)** (He et al., 2022) apply a masking-and-reconstruction pretext task to images.

Definition

MAE randomly masks a very large fraction (e.g., 75%) of an image’s patches. A ViT **encoder** processes *only the small number of visible (unmasked) patches* (a major efficiency advantage, since self-attention’s cost scales with sequence length). A lightweight **decoder** then takes the encoded visible patches plus mask tokens (placeholder embeddings marking masked positions) and is trained to reconstruct the raw pixel values of the masked patches.

Key Idea

Masking such a high fraction of patches is deliberate and important: with images (unlike text), nearby pixels are highly redundant, so masking only a small fraction (as BERT does with words, around 15%) would make the reconstruction task too easy to solve using simple local interpolation, without requiring the network to learn genuinely high-level semantic understanding. A very high masking ratio forces the encoder to build a representation rich enough to support reconstructing large

missing regions purely from sparse visible context.

3.6 DINO: Self-Distillation with No Labels

DINO (Caron et al., 2021) trains a student ViT to match the output of a momentum-updated teacher ViT, applied to different augmented crops of the same image (architecturally similar in spirit to BYOL), using a self-distillation cross-entropy loss with careful centering and sharpening to avoid collapse. A striking empirical finding: DINO’s self-attention maps, with no supervision or labels whatsoever, spontaneously learn to focus sharply on the semantic foreground objects in an image – a property exploited directly by the interpretability technique in Section 5.

3.7 Comparative Summary

Method	Pretext task	Key mechanism	Negatives?
SimCLR	Contrastive (views)	Large batch negatives, NT-Xent	Yes
MoCo	Contrastive (views)	Momentum encoder + queue	Yes
BYOL	Predict target output	Momentum target + predictor	No
MAE	Masked patch re-con.	Encode visible patches only	No
DINO	Self-distillation	Student/teacher (momentum) ViT	No

4 Multimodal and Foundation Vision Models: CLIP and Foundational Image Pre-training

4.1 Foundation Models: The General Paradigm

Definition

A **foundation model** is a model pretrained on broad, large-scale data, designed to be **adapted** (via fine-tuning, linear probing, prompting, or zero-shot transfer) to a wide range of downstream tasks, rather than being trained for one specific task from the start. The self-supervised methods of Section 3 already embody part of this idea (learning general-purpose features from broad, unlabeled image data); this section studies models that push the paradigm further by incorporating **natural language** as a second modality.

4.2 CLIP: Contrastive Language-Image Pretraining

CLIP (Radford et al., 2021) learns a *joint* embedding space for images and text by training on hundreds of millions of (image, caption) pairs collected from the web, using a contrastive objective directly analogous to Section 3’s contrastive methods, but *across modalities* rather than across augmented views of the same image.

4.2.1 Architecture

- An **image encoder** (a ViT or a CNN such as ResNet) maps an image to an embedding $v \in \mathbb{R}^d$.
- A **text encoder** (a Transformer, as in Module 3) maps a caption to an embedding $t \in \mathbb{R}^d$.
- Both embeddings are projected into the *same* shared d -dimensional space and L_2 -normalized.

4.2.2 Training Objective

Definition

Given a batch of N (image, text) pairs, CLIP computes the cosine similarity between every image embedding and every text embedding in the batch, forming an $N \times N$ similarity matrix. It is trained with a symmetric contrastive loss (InfoNCE-style, directly analogous to SimCLR’s NT-Xent loss in Section 3) that maximizes the similarity of each image with its *correct, matching* caption (the diagonal of the matrix) relative to all $N - 1$ *other, non-matching* captions in the batch (the off-diagonal entries), and symmetrically for each caption relative to all other images.

Key Idea

CLIP’s key insight is that natural-language captions provide a vastly richer and more flexible source of supervision than a fixed, predetermined set of class labels (as in traditional supervised ImageNet pre-training): captions can describe virtually unlimited visual concepts, attributes, and relationships, in whatever vocabulary happens to appear in web text, rather than being restricted to (for example) exactly 1000 ImageNet categories decided in advance by the dataset’s creators.

4.2.3 Zero-Shot Classification

Key Idea

Because CLIP’s image and text embeddings share a common space, it can classify images into *any* set of candidate classes **without any task-specific fine-tuning**: construct a text prompt for each candidate class (e.g., “a photo of a {class name}”), embed all candidate prompts with the text encoder, embed the query image with the image encoder, and predict the class whose text embedding has the highest cosine similarity to the image embedding. This **zero-shot transfer** capability – performing a new classification task specified only by a list of class names, with no labeled training examples for that specific task – was one of CLIP’s most striking and influential results, showing competitive performance with fully-supervised models trained specifically on some of these benchmarks.

Example 1. To classify an image as one of {“cat”, “dog”, “car”} using

CLIP with no fine-tuning: form the prompts “a photo of a cat”, “a photo of a dog”, “a photo of a car”; compute their text embeddings t_1, t_2, t_3 and the image’s embedding v ; predict $\arg \max_i \cos(v, t_i)$. If $\cos(v, t_1) = 0.31$, $\cos(v, t_2) = 0.24$, $\cos(v, t_3) = 0.08$, the predicted class is “cat”. Note that entirely new candidate classes could be added at inference time simply by writing a new prompt, with no retraining required.

4.3 Foundational Image Pretraining

CLIP is one prominent instance of a broader trend of training large **vision foundation models** on internet-scale data, using a variety of pretraining recipes, each producing representations with somewhat different strengths:

- **Supervised pretraining** (e.g., classic ImageNet-1k pretrained CNNs/ViTs): strong, well-understood features, but tied to the specific label vocabulary used during training.
- **Self-supervised pretraining** (Section 3: SimCLR, MoCo, BYOL, MAE, DINO): learns from unlabeled images alone; DINOv2 (Oquab et al., 2023) scaled this recipe substantially, producing general-purpose visual features that transfer well to dense prediction tasks (depth estimation, segmentation) even *without* any fine-tuning, via simple linear probes.
- **Contrastive language-image pretraining** (CLIP and successors, e.g., ALIGN, OpenCLIP): learns a joint vision-language space directly from naturally-occurring web image-text pairs, enabling zero-shot transfer and serving as a foundation for later multimodal systems.
- **Promptable foundation models** (e.g., the Segment Anything Model, SAM, Kirillov et al., 2023): trained on a very large, diverse segmentation dataset to segment *any* object given a prompt (a point, box, or rough mask) at inference time, generalizing to novel objects and image types not seen during training.

Key Idea

A unifying theme across all of these is the same **pretrain-then-adapt** paradigm introduced at the start of this section: invest heavily in a single, broad, large-scale pretraining phase, and then adapt the resulting representation cheaply (via zero-shot prompting, linear probing, or lightweight fine-tuning) to many different downstream tasks – rather than training a separate, task-specific model from scratch for every new

application, which is far more data- and compute-expensive.

5 Attention Rollout

Having studied how ViTs are trained (Sections 2–4), we now turn to **interpreting** what a trained ViT has actually learned to attend to – specifically, which input image patches most influence its final prediction.

5.1 Why Raw Attention Maps Are Insufficient

Key Idea

A natural first idea is to simply visualize the attention weights from the *final* layer’s [CLS] token row: this directly shows which patches the last layer attended to. However, this ignores everything that happened in *earlier* layers: information reaching the final layer’s attention computation has already been mixed and re-mixed across all previous layers (and residual connections additionally let a token’s original identity persist alongside whatever attention has mixed in). A patch that was highly influential in shaping an intermediate representation two layers back, but is not directly attended to by the very last layer, would be invisible in a raw last-layer attention map, even though it substantially influenced the final output.

5.2 The Attention Rollout Method

Attention Rollout (Abnar & Zuidema, 2020) addresses this by tracking how attention (and the residual/skip-connection pathway) propagates information *across all layers*, from the input patches all the way to the final representation.

Definition

For each layer ℓ , let $A_\ell \in \mathbb{R}^{N \times N}$ be the attention matrix (averaged across heads), where $A_\ell[i, j]$ is the attention weight from token i to token j . To account for the residual connection at each layer (which lets a token’s representation flow forward *unchanged*, in addition to whatever it gathers via attention), define the **augmented attention matrix**:

$$\hat{A}_\ell = 0.5A_\ell + 0.5I$$

where I is the identity matrix, and each row is re-normalized to sum to 1. **Attention Rollout** is then computed as the matrix product of

these augmented attention matrices across all L layers:

$$\text{Rollout} = \hat{A}_L \cdot \hat{A}_{L-1} \cdots \hat{A}_1$$

Key Idea

The $0.5A_\ell + 0.5I$ construction directly models the residual connection: with probability (weight) 0.5, a token’s information at this layer is a function of its attention-weighted mixture with other tokens (A_ℓ); with the other 0.5, it simply passes through unchanged (the identity term). Multiplying these augmented matrices together, layer by layer, tracks how “credit” or information flow from every original input patch is diluted, mixed, and propagated forward through the *entire* depth of the network – rather than looking at any single layer in isolation. The row of the final Rollout matrix corresponding to the [CLS] token gives a distribution over input patches representing their aggregate contribution to the final classification decision, which can be reshaped back into the image’s spatial grid and visualized as a heatmap overlaid on the original image.

5.3 Worked Example

Example 2. Consider a toy 2-layer, 2-token ([CLS] and one patch) Transformer for simplicity. Suppose the raw attention matrices (rows sum to 1, row/column order is [CLS, patch]) are:

$$A_1 = \begin{pmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{pmatrix}, \quad A_2 = \begin{pmatrix} 0.5 & 0.5 \\ 0.2 & 0.8 \end{pmatrix}$$

First, augment each with the identity and re-normalize ($0.5A_\ell + 0.5I$, rows already sum to 1 since A_ℓ ’s rows do and I ’s rows do):

$$\hat{A}_1 = 0.5 \begin{pmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{pmatrix} + 0.5 \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0.8 & 0.2 \\ 0.15 & 0.85 \end{pmatrix}$$

$$\hat{A}_2 = 0.5 \begin{pmatrix} 0.5 & 0.5 \\ 0.2 & 0.8 \end{pmatrix} + 0.5 \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0.75 & 0.25 \\ 0.1 & 0.9 \end{pmatrix}$$

Computing $\text{Rollout} = \hat{A}_2 \cdot \hat{A}_1$:

$$\begin{aligned} \text{Rollout} &= \begin{pmatrix} 0.75 & 0.25 \\ 0.1 & 0.9 \end{pmatrix} \begin{pmatrix} 0.8 & 0.2 \\ 0.15 & 0.85 \end{pmatrix} \\ &= \begin{pmatrix} (0.75)(0.8) + (0.25)(0.15) & (0.75)(0.2) + (0.25)(0.85) \\ (0.1)(0.8) + (0.9)(0.15) & (0.1)(0.2) + (0.9)(0.85) \end{pmatrix} \\ &= \begin{pmatrix} 0.6375 & 0.3625 \\ 0.215 & 0.785 \end{pmatrix} \end{aligned}$$

Reading the $[CLS]$ row (first row): the final classification decision’s rollout attention attributes 63.75% of its aggregate attention to the $[CLS]$ token itself (i.e., its own persisting identity via residual connections) and 36.25% to the single patch token, across both layers combined.

5.4 Relation to Other Interpretability Methods

- **Raw last-layer attention:** Simpler, but (as discussed above) ignores multi-layer information mixing and residual pathways.
- **Gradient-based saliency methods** (e.g., Grad-CAM, originally developed for CNNs): Use gradients of the output with respect to intermediate activations to estimate the importance of different spatial regions; conceptually different from attention rollout, which relies purely on the (already-computed) attention weights themselves, without needing an additional backward pass.
- **DINO’s emergent attention maps** (Section 3.6): A related but distinct phenomenon – some self-supervised ViTs’ *raw* last-layer attention maps happen to already highlight semantic objects well, without needing the rollout procedure, though this is an emergent property of the specific training objective (DINO) rather than a general guarantee for any ViT.

Key Idea

A key caveat, and an active topic of debate in interpretability research, is that attention weights (rolled out or otherwise) are not necessarily a fully faithful causal explanation of *why* the network produced a particular output – high attention to a region indicates that information from that region was mixed in heavily, but does not, on its own, prove that this information was the *decisive* factor in the final decision. Attention rollout should be understood as a useful, principled visualization

of information flow through the network's actual computation graph, rather than a definitive causal explanation.

6 Applications and Challenges of Advanced Visual Representation Learning

6.1 Applications

- **Zero-shot and open-vocabulary recognition:** CLIP-style models enable classification, retrieval, and (when combined with detection/segmentation architectures) **open-vocabulary object detection and segmentation** – recognizing object categories specified only by a text description at inference time, rather than a fixed, pre-trained label set.
- **Image-text retrieval and search:** Joint embedding spaces (CLIP and successors) directly support searching a large image database using a natural-language text query, or vice versa, by nearest-neighbor lookup in the shared embedding space.
- **Multimodal generation:** Text-to-image diffusion models (Module 5) are commonly *conditioned* on CLIP text embeddings, directly connecting this module’s multimodal representations to generative modeling.
- **Data-scarce domains:** Self-supervised pretraining (Section 3) is especially valuable in domains such as medical imaging or satellite/remote sensing, where large labeled datasets are expensive or impossible to obtain, but large amounts of unlabeled imagery exist.
- **Dense prediction transfer:** Foundation models such as DINOv2 provide general-purpose features that transfer well, often via simple linear probes, to depth estimation, semantic segmentation, and other pixel-level tasks, without task-specific pretraining.
- **Promptable segmentation:** The Segment Anything Model (SAM) and similar systems allow segmenting arbitrary, previously unseen object categories given only a simple prompt (point, box, or rough mask) at inference time.

6.2 Core Challenges

- **Data and compute hunger:** As discussed in Section 1, ViTs trained from scratch typically require substantially more data (and/or specialized training recipes like DeiT’s distillation) than CNNs to reach comparable performance, and foundation-model-scale pretraining (CLIP, DINOv2) requires web-scale data and very large compute budgets, which are inaccessible to most individual researchers or organizations.

- **Quadratic attention cost limits resolution:** Self-attention’s $O(N^2D)$ cost (Module 3, Section 2) directly limits the number of patches (and thus image resolution) that can be practically processed, motivating research into more efficient attention mechanisms (Module 3’s specialized attention variants) for high-resolution vision tasks.
- **Data quality and bias:** Web-scraped image-text datasets (used to train CLIP and similar models) are uncured and can contain harmful stereotypes, offensive content, and skewed representation of different demographics or cultures, which the resulting models can inherit and reproduce in their downstream behavior.
- **Evaluation and contamination concerns:** Because foundation models are pretrained on enormous, often loosely-documented web-scraped datasets, it can be difficult to be fully confident that a “zero-shot” evaluation benchmark’s images (or very similar ones) were not already present in the pretraining data, which would inflate reported zero-shot performance relative to genuinely novel data.
- **Interpretability remains partial:** While Attention Rollout (Section 5) provides a principled, well-motivated view into information flow, it is not a complete or fully causal explanation of model behavior, and interpretability for large, complex vision (and vision-language) foundation models remains an open, actively-researched problem.
- **Robustness and spurious correlations:** Learned visual representations can latch onto spurious correlations present in training data (e.g., associating an object with a typical background context) rather than the causally-relevant visual features, which can hurt generalization to distribution shifts, unusual contexts, or adversarially-crafted inputs.
- **Variable resolution and positional embedding limitations:** ViT’s learned positional embeddings are tied to a specific number of patches (a specific training resolution); applying a pretrained ViT to images of a different resolution requires interpolating the positional embeddings, which can degrade performance if not done carefully.

Key Idea

A recurring theme across these challenges is that the same properties that make foundation vision models powerful – learning from vast, loosely-curated, web-scale data with minimal hand-designed structure – are also the direct source of their biggest risks (inherited bias, evalu-

ation contamination, limited interpretability, data/compute cost). Advancing this field responsibly requires progress not just on raw benchmark performance, but on data curation, evaluation rigor, efficiency, and interpretability in tandem.

REFERENCES

- Prince, Simon JD. Understanding deep learning. MIT press, 2023.
- Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). Deep learning (Vol. 1, No. 2, pp. 1-800). Cambridge: MIT press. <https://www.deeplearningbook.org/>
- Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Vol. 1. Cham, Switzerland: Springer, 2024.
- Torralba, A., Isola, P., & Freeman, W. T. (2024). Foundations of computer vision. MIT Press.